

National College of Ireland

Project Submission Sheet

Swe Swe Aung

Student Name:

25167219

Student ID:

2026

Programme:HDSDADINTJAN26I..... **Year:**

Databases for Analytics

Module: (H8DFA).....

Jaswinder Singh

Lecturer:

Submission Due Date: 25 July 2026.....

Project Title: Design and Implementation of a Database for FastLink 3PL Pharmacy Logistics

Word Count:

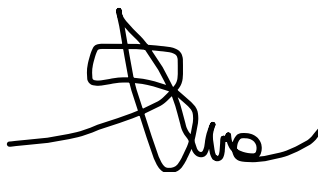
Academic Integrity Certificate Verification

Code:pLWgdWDsnB.....

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting Monday, 19th January 2026, all students are required to successfully complete the Epigeum Academic Integrity Course, which is available on Moodle. This is a self-paced online course consisting of 4 modules and will take approximately 2 hours to complete. Once you have successfully completed the course, you will receive a Certificate & Verification Code. This code must be included on the Project Cover Sheet when submitting your work. For full details and guidance, please visit: <https://libguides.ncirl.ie/academicintegrity/epigeumforstudents>



Signature
e:

Date:7/25/2026.....
.....

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.

3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplements in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date
Swe Swe Aung	Databases for Analytics (H8DFA)	7/25/2026

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click [here](#).

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool
Microsoft Word File	Grammar and punctuation checks	Microsoft Word Automative Tool
Claude (Anthropic)	Understanding the assignment, brainstorming the scenario, explaining database concepts (normalisation, indexing), checking SQL for errors, and feedback on how to structure and word explanations	https://claude.ai/

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used.**

Which columns should I index in my database, and why?	Suggested indexing columns used for filtering and joining (such as shipment_status and supplier_id), explained why primary keys are already indexed automatically, and noted that on small data the planner may still use a sequential scan. I chose the final indexes and wrote the justification myself.
Check my SQL DDL for errors.	Pointed out a foreign-key referencing a column that didn't exist and a data-type issue. I fixed these myself in my script.
Generate realistic sample data for my pharmaceutical logistics database that follows my business rules	

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

Which entity are you asking about? We've defined attributes for all 11 entities — let me know which one you want me to explain:

- Or say "all" and I'll go through every single one.

1. In the medicine table, remove the manufacturer
2. In proof of delivery, remove delivery_date
3. Explain these : `pod_statusSent / Acknowledged / Failed`
4. I think that we should remove `min_required_temp` and `max_required_temp` from `temperatureRecord`. To know the status calculate it from the medicine table

Thought process

1. Remove manufacturer from Medicine ✓

2. Remove delivery_date from ProofOfDelivery ✓

- Customer (30): mix of Business (hospitals, pharmacies) and Individual; few Inactive. Orders only come from Active customers.
- Supplier (15): mostly Active, one Suspended, one Inactive — so we can show the “no suspended supplier” rule holds.
- SupplierCapability (~39): transport + storage capabilities; only some suppliers have **Cold Storage**.
- Medicine (25): ~6 cold-chain (Insulin, vaccines, albumin — 2–8 °C) and ambient.
- Orders (60): spread Jan–Jul 2026.
- Shipment (80): each assigned to a non-suspended supplier; any shipment carrying a cold medicine goes only to a **Cold Storage**-capable supplier.
- ShipmentItem (~90): each shipment has ≥1 medicine.
- Invoice (80): one per shipment; **due_date** auto = invoice_date + 15.
- TemperatureRecord (~49): only for cold shipments; a few readings deliberately **Out of Range** so your compliance queries have something find.
-

1. FastLink 3PL Pharmacy Logistics
2. transportation_type

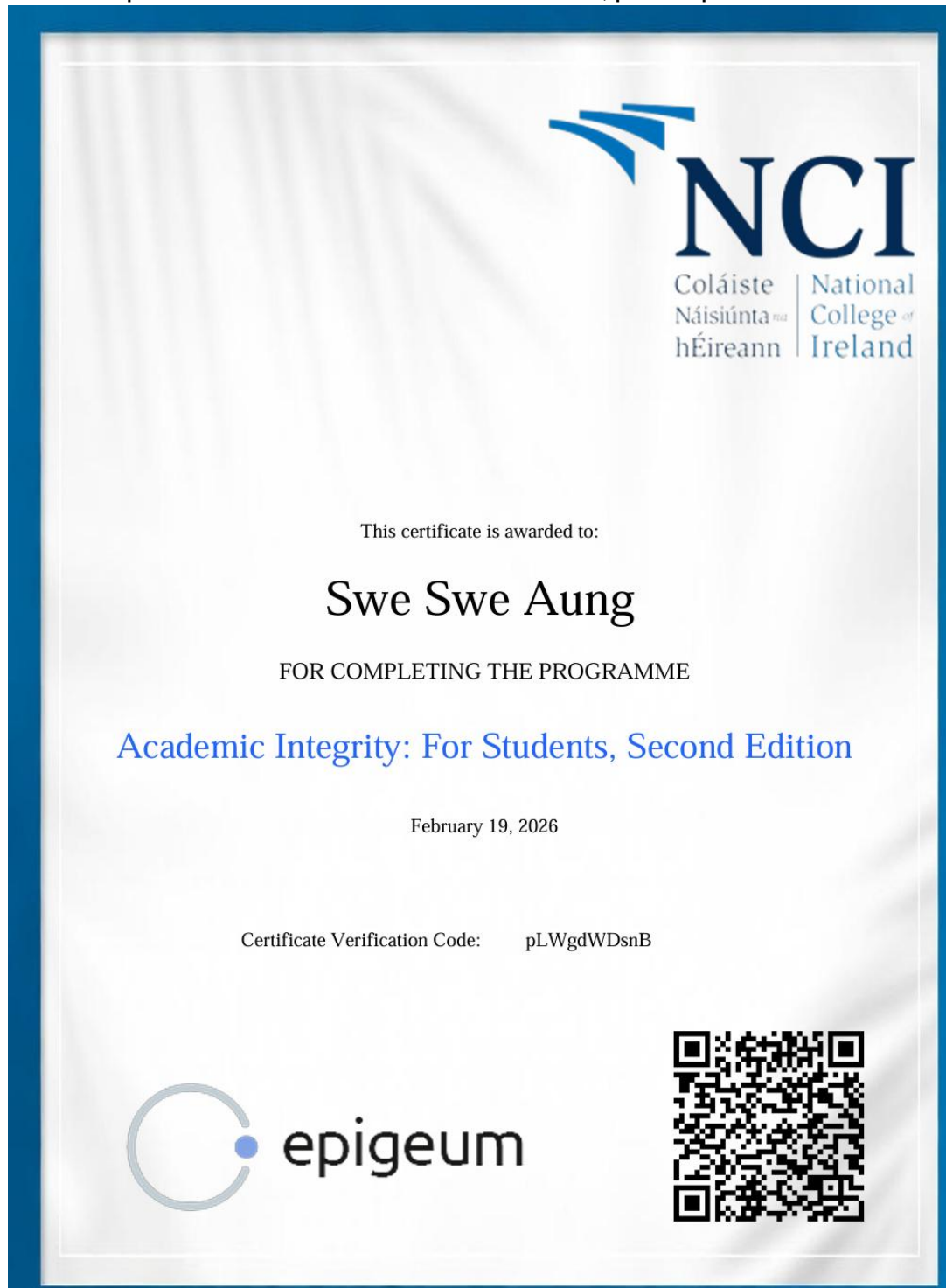
Jul 18, 2:52 PM

API Error: Response stalled mid-stream. The response above may be incomplete.

API Error: 529 Overloaded. This is a server-side issue, usually temporary — try again in a moment. If it persists, check <https://status.claude.com>.

Academic Integrity Acknowledgement Supplement

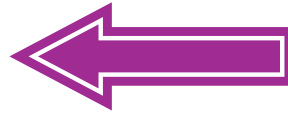
The red square below indicates certificate number, please put this in the above field



Academic Integrity Subject Guide or how to access the site:

<https://libguides.ncirl.ie/academicintegrity/epigeumforstudents>

**Completion of Epigeum Academic Integrity Online Course is Mandatory for
all registered students of NCI**



Project Title: Design and Implementation of a Database for
FastLink 3PL Pharmacy Logistics

Domain: Transportation and Logistics System

Student Name: SWE SWE AUNG

Student ID: 25167219

Phase 1: Conceptual and Logical Database Design

Task 1: Scenario Description and Business Rules

This report describes the design and implementation of a database for a pharmaceutical logistics company, and also covers the structure (DDL), data population (DML), and reporting queries. In this report, the company's name is FastLink 3PL Pharmacy Logistics, which is also a third-party logistics provider and transports pharmaceutical products with the support of the suppliers to healthcare customers across Ireland, under GDP (Good Distribution Practice) cold-chain rules.

A logistics information system helps third-party logistics providers manage their work and improve performance [1]. This idea inspired the assignment, so the FastLink 3PL database was built as a logistics information system for a third-party provider. The FastLink scenario was also based on Uniphar, an actual Irish company providing third-party logistics and cold chain storage services for medicines [2].

The main purposes of the database are to record the orders coming from the customer and the shipments to deliver these orders, the temperature conditions, and the invoices that charge the customer for their shipments. The stakeholders are customers, such as hospitals, pharmacies, and individuals; logistics staff who assign shipments and monitor cold-chain compliance; and suppliers with certified transport and storage capabilities, and the finance team who handle invoicing.

The key data that needs to be stored in the database are customer information and their status, suppliers' details and their capabilities, medicine information and their storage conditions (Ambient 15–25 °C, Cold 2–8 °C, Frozen –25 to –15 °C), order information with special notes, and shipment information, the medicines on each shipment; temperature readings; and invoices.

The main transactions of this database are that customers will place orders, and these orders will be stored in the orders table. The logistics staff will create one or more shipments per order and assign them to the compliance suppliers and store them in the shipments. From the order, if new medicines show up, then add them to the medicine table. From the shipment, the quantity of the medicines will be added to the shipment item table. In the temperature record table, temperature readings will be added to it, and based on the readings, flag the temperature status. For all shipments, invoices are issued, and the payment status is updated every day.

There are 13 business rules set up in this database. They are:

1. Each order must be registered with one and only one customer.
2. One order may create one or many shipments to support deliveries.
3. Each shipment must be assigned to exactly one supplier, and that supplier cannot be flagged as suspended status at the time of assignment. (Task 11)
4. All suppliers must have at least one capability and must be registered in the SupplierCapability table before the supplier can be assigned to any shipment.
5. Each shipment must contain at least one medicine item, and the quantity of each medicine must be recorded in the ShipmentItem entity.
6. Each medicine item must be assigned to a medicine category and exactly one storage condition. There are Ambient (15–25°C), Cold (2–8°C), or Frozen (-25°C to -15°C). The corresponding minimum and maximum storage temperatures must be recorded in the system. (Task12)
7. Each temperature record must be checked against the required range of medicines carried, and any reading outside it must be flagged as Out of Range. This is important for GDP compliance. (Task14)
8. Each shipment of cold/frozen medicine can only be assigned to a supplier who has a Supplier Capability that matches the type of storage (Cold Storage for cold, Frozen for frozen). (Task 13)
9. Temperature records must be logged regularly for every shipment carrying cold or frozen medicines, and each record must store a timestamp, temperature value, and status. (Task 14)

10. Each shipment must have one and only one invoice that records the total amount, due date, and payment status.
11. Each invoice's payment due date must be automatically set to 15 days after the invoice date.
12. The payment status of an invoice must be automatically updated to Overdue when payment has not been received by the due date.
13. Each shipment must record the transportation type to know how many of each type have been delivered. (Task 11)

Task 2: Entity Relationship Diagram

Figure 1 shows the Entity Relationship Diagram (ERD) for the FastLink 3PL Pharmacy Logistics database. It is based on the business rules stated in Task 1 and displays all nine entities, their attributes, primary keys (PK), foreign keys (FK), and the cardinality between the entities. Crow's Foot notation is used where "many" cardinality is represented by a crow's foot and "one" cardinality by one line.

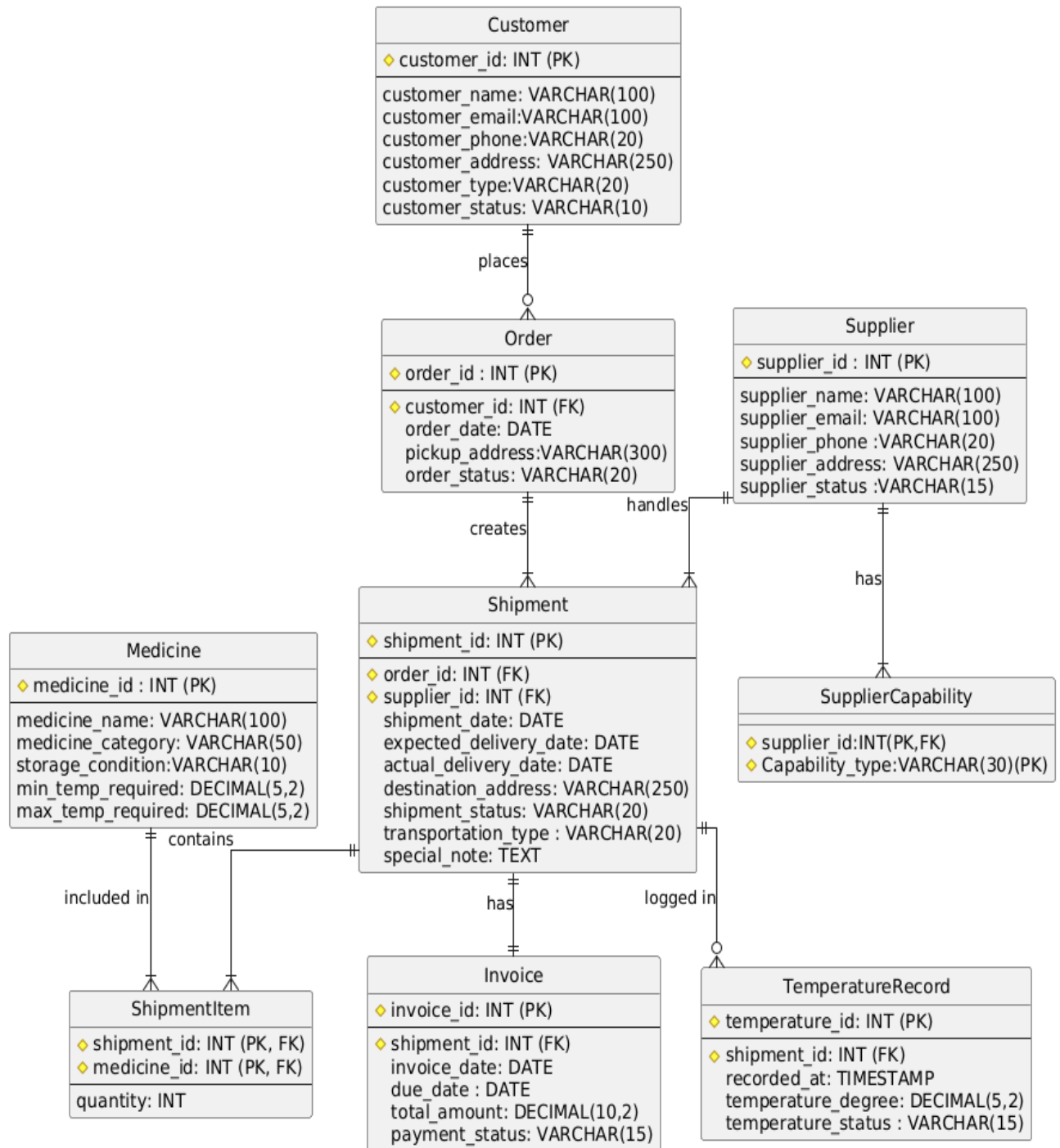


Figure 1: ER Diagram for FastLink Pharmacy Logistics

The diagram contains nine entities: Customer, Order, Supplier, SupplierCapability, Medicine, Shipment, ShipmentItem, Invoice, and TemperatureRecord. The main relationships are:

- One customer can place many orders, but one order belongs to only one customer.
- One order can generate many shipments, but each shipment belongs to only one order.
- One supplier can handle many shipments, but each shipment is assigned to only one supplier.
- One supplier may have one or many capabilities. These are recorded in the SupplierCapability table.
- One medicine can appear on many shipments through shipment_item.
- Shipment is the central table, with relationships to many tables such as Order, Supplier, Invoice, Medicine, and TemperatureRecord.
- One shipment can contain many medicines, and one medicine can be in many shipments.
- One shipment has exactly one invoice. This is a one-to-one relationship.
- One shipment can be logged in many temperature records, but each reading record belongs to one shipment.

SupplierCapability and ShipmentItem are linking/bridge tables, which consist of composite keys built using foreign keys. These resolve the many-to-many relationships in the design.

Derived Attributes

Two derived attributes are defined for the design. They will not be stored physically in the database since they can be computed when required.

The first attribute is delivery_duration, which is the number of days that a delivery takes place. This is computed as actual_delivery_date - shipment_date. This attribute is not stored in the physical database since it can always be recomputed from two other existing attributes in the Shipment table, because if one of these two values changes in the future, inconsistency might arise.

The second attribute is temperature_compliance_rate, which is the percentage of the number of temperature readings within the required range and the total number of readings. It is computed as (number of Within Range readings ÷ total readings) × 100. It is not stored because it can be calculated from the temperature readings whenever it is needed. Therefore, it is always up to date.

Task 3: Relational Model and Normalisation

An ER diagram shows the design of the database. A relational database stores data in tables [3].

customer(**customer_id**, customer_name, customer_email UQ, customer_phone, customer_address, customer_type, customer_status)

The customer table in this database is the reference table. It stores information about an individual or organization that makes orders from FastLink. The customer_id is the primary key (unique ID for each customer). The customer_email attribute is also a unique key; therefore, no two customers can share an email address. All attributes are dependent on the primary key. There are no foreign keys in this table because a customer doesn't depend on any other table.

orders(**order_id**, customer_id FK → CUSTOMER.customer_id, order_date, pickup_address, order_status)

This table stores the orders placed by customers. The order_id is the primary key. The customer_id is a foreign key linking each order to each customer. One customer can have many orders, but each order belongs to only one customer. All other attributes depend on the primary key.

shipment(**shipment_id**, order_id FK → ORDERS.order_id, supplier_id FK → SUPPLIER.supplier_id, shipment_date, expected_delivery_date, actual_delivery_date, destination_address, shipment_status, transportation_type, special_note)

This table stores each delivery that is made from an order. The shipment_id is the primary key. It has two foreign keys: order_id and supplier_id. One order can have many shipments, and one supplier can handle many shipments. All other attributes depend on the primary key.

supplier(**supplier_id**, supplier_name, supplier_email UQ, supplier_phone, supplier_address, supplier_status)

The supplier table is a reference table. It stores information about the carriers that transport shipments for FastLink. The supplier_id is the primary key. The supplier_email is a unique key, so no two suppliers can share an email. All attributes depend on the primary key. There are also no foreign keys.

```
**supplier_capability**(**supplier_id** FK → SUPPLIER.supplier_id, **capability_type**,  
PK(supplier_id, capability_type))
```

This is a bridge table. It stores the capabilities of each supplier, such as Road, Cold Storage, or Frozen Storage, etc. The primary key is two columns together, supplier_id and capability_type, so one supplier can have many capabilities but cannot repeat the same one. The supplier_id is a foreign key linking to the supplier table.

```
**medicine**(**medicine_id**, medicine_name, medicine_category, storage_condition,  
min_temp_required, max_temp_required)
```

This table is a reference table. It stores the products' details and their storage needs. The medicine_id is the primary key. The storage_condition shows if a medicine is Ambient, Cold, or Frozen, and the two temperature columns hold its allowed range. All attributes depend on the primary key. There are also no foreign keys.

```
**shipmentitem**(**shipment_id** FK → SHIPMENT.shipment_id, **medicine_id** FK →  
MEDICINE.medicine_id, quantity, PK(shipment_id, medicine_id))
```

This is a bridge table. It stores which medicines are on each shipment and how many. The primary key is the two columns together, shipment_id and medicine_id. Both are also foreign keys. The quantity depends on both key columns together, because the same medicine can appear on different shipments in different amounts.

```
**invoice**(**invoice_id**, shipment_id FK → SHIPMENT.shipment_id UQ, invoice_date,  
due_date, total_amount, payment_status)
```

This table stores the bill for each shipment. The invoice_id is the primary key. The shipment_id is a foreign key and is also unique in this table. So, each shipment has exactly one invoice. The due_date is set automatically to 15 days after the invoice date. All other attributes depend on the primary key.

temperaturerecord(**temperature_id**, shipment_id FK → SHIPMENT.shipment_id, recorded_at, temperature_degree, temperature_status)

This table stores the temperature readings taken with cold or frozen shipments. The temperature_id is the primary key. The shipment_id is a foreign key, so one shipment can have many reading records. The temperature_status shows whether a reading is Within Range or Out of Range. All other attributes depend on the primary key.

Key Functional Dependencies

A functional dependency $A \rightarrow B$ implies that for any value of A, there can be one and only one value of B. Some of the functional dependencies of this design include the following:

- customer_id → customer_name, customer_email, customer_phone, customer_address, customer_type, customer_status
- supplier_id → supplier_name, supplier_email, supplier_phone, supplier_address, supplier_status
- medicine_id → medicine_name, medicine_category, storage_condition, min_temp_required, max_temp_required
- order_id → customer_id, order_date, pickup_address, order_status
- shipment_id → order_id, supplier_id, shipment_date, expected_delivery_date, actual_delivery_date, destination_address, shipment_status, transportation_type, special_note
- (shipment_id, medicine_id) → quantity
- invoice_id → shipment_id, invoice_date, due_date, total_amount, payment_status
- temperature_id → shipment_id, recorded_at, temperature_degree, temperature_status

The determinant of the primary key on the left side of each table determines all other attributes of that table. For the shipment_item table, the determinant is the entire composite key (shipment_id, medicine_id). That is why quantity is fully functionally dependent on the whole key.

Normalisation to Third Normal Form (3NF)

Normalisation organises data in separate, well-structured tables to avoid repetitions and simplify updating of the database. The database design was already checked up to 3NF.

First Normal Form (1NF)

The First Normal Form (1NF) is a relation where the intersection of each row and column has exactly one value with no repeating groups. All tables in this database comply with 1NF.

A supplier's capabilities are not stored as a repeating group in one row; each capability is a separate row in `supplier_capability`. And the medicines on a shipment are separate rows in `shipment_item`, not repeating columns. Every row and column contains one and only one value.

Second Normal Form (2NF)

The Second Normal Form (2NF) is a relation in 1NF, and all the non-primary key attributes have full functional dependencies on the primary keys. This condition is checked for the two tables that have composite keys. In `shipment_item`, the quantity attribute is fully dependent on the whole key (`shipment_id`, `medicine_id`), not on part of it, so there is no partial dependency. There are no non-key attributes in `supplier_capability`, so a partial dependency cannot exist. All other tables have a single-column primary key, which cannot have a partial dependency. Therefore, all tables are in 2NF.

Third Normal Form (3NF)

3NF is a relation that is in 2NF and has no transitive dependency, meaning no non-primary-key attribute depends on another non-primary-key attribute. All tables satisfy this. In each table, all the descriptive facts about an entity are stored along with the entity only. The address of a customer is stored in the customer table and linked with orders through the `customer_id` column, and not in the order table. Similarly, the status of a supplier is stored in the supplier table and not in the shipment table.

Removing update anomalies

There will be no update anomalies since data is not repeated across tables. New records can be inserted without duplicating any other information, no data loss while deleting a record, and changing a value is done in just one place. Therefore, this database model avoids insertion, deletion, and modification anomalies.

Phase 2: Physical Database Implementation

Task 4: SQL DDL Schema Creation

The schema was implemented in PostgreSQL with Data Definition Language DDL, where nine tables were created with the help of primary keys, foreign keys, and other constraints.

Learning and practicing the SQL syntax and features was from a PostgreSQL Tutorial, which is an online learning site used to reference this project's SQL [4].

```
CREATE TABLE Customer (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL,  
    customer_email VARCHAR(100) NOT NULL UNIQUE,  
    customer_phone VARCHAR(20)not null,  
    customer_address VARCHAR(250) NOT NULL,  
    customer_type VARCHAR(20) NOT NULL CONSTRAINT customer_type_check CHECK  
(customer_type IN ('Individual', 'Business')),  
    customer_status VARCHAR(10) NOT NULL CONSTRAINT customer_status_check CHECK  
(customer_status IN ('Active', 'Inactive'))  
);  
  
CREATE TABLE Supplier (  
    supplier_id INT PRIMARY KEY,  
    supplier_name VARCHAR(100) NOT NULL,  
    supplier_email VARCHAR(100) NOT NULL UNIQUE,  
    supplier_phone VARCHAR(20)NOT NULL,  
    supplier_address VARCHAR(250) NOT NULL,  
    supplier_status VARCHAR(15) NOT NULL CONSTRAINT supplier_status_check CHECK  
(supplier_status IN ('Active', 'Inactive', 'Suspended'))  
);
```

```

CREATE TABLE SupplierCapability (
    supplier_id INT NOT NULL,
    capability_type VARCHAR(30) NOT NULL CONSTRAINT capability_type_check CHECK
(capability_type IN (
        'Road', 'Air', 'Sea', 'Rail',
        'Ambient Storage', 'Cold Storage',
        'Frozen Storage'
    )),
    PRIMARY KEY (supplier_id, capability_type),
    FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)
);

```

```

CREATE TABLE Medicine (
    medicine_id INT PRIMARY KEY,
    medicine_name VARCHAR(100) NOT NULL,
    medicine_category VARCHAR(50) NOT NULL,
    storage_condition varchar(10) not null
    constraint storage_condition_check check(storage_condition in
('Ambient','Cold','Frozen')),
    min_temp_required DECIMAL(5,2) not null,
    max_temp_required DECIMAL(5,2) not null,
    CONSTRAINT temp_required_check CHECK
(min_temp_required<max_temp_required),
    CONSTRAINT storage_range_check check ((storage_condition='Ambient' and
min_temp_required=15 and max_temp_required=25)
    or (storage_condition='Cold' and min_temp_required=2 and max_temp_required=8)
    or (storage_condition='Frozen' and min_temp_required=-25 and
max_temp_required=-15))

```

);

CREATE TABLE Orders (

order_id INT PRIMARY KEY,

customer_id INT NOT NULL,

order_date DATE NOT NULL,

pickup_address VARCHAR(300),

order_status VARCHAR(20) not null constraint order_status_check check(order_status in ('Pending', 'Processing', 'Completed', 'Cancelled'))),

FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)

);

CREATE TABLE Shipment (

shipment_id INT PRIMARY KEY,

order_id INT NOT NULL,

supplier_id INT NOT NULL,

shipment_date DATE NOT NULL,

expected_delivery_date DATE NOT NULL,

actual_delivery_date DATE,

destination_address VARCHAR(250) NOT NULL,

shipment_status VARCHAR(20) NOT NULL CONSTRAINT shipment_status_check CHECK (shipment_status IN ('Pending', 'In Transit', 'Delivered', 'Failed')),

transportation_type VARCHAR(20) NOT NULL CONSTRAINT transportation_type_check CHECK (transportation_type IN ('Road', 'Air', 'Sea', 'Rail')),

special_note TEXT,

FOREIGN KEY (order_id) REFERENCES Orders(order_id),

FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)

);

```

CREATE TABLE ShipmentItem (
    shipment_id INT NOT NULL,
    medicine_id INT NOT NULL,
    quantity INT NOT NULL CONSTRAINT quantity_check CHECK (quantity > 0),
    PRIMARY KEY (shipment_id, medicine_id),
    FOREIGN KEY (shipment_id) REFERENCES Shipment(shipment_id),
    FOREIGN KEY (medicine_id) REFERENCES Medicine(medicine_id)
);

CREATE TABLE Invoice (
    invoice_id INT PRIMARY KEY,
    shipment_id INT NOT NULL UNIQUE,
    invoice_date DATE NOT NULL,
    due_date DATE GENERATED ALWAYS AS (invoice_date + INTERVAL '15 days') STORED,
    total_amount DECIMAL(10,2) NOT NULL CONSTRAINT total_amount_check CHECK
(total_amount > 0),
    payment_status VARCHAR(15) NOT NULL CONSTRAINT payment_status_check CHECK
(payment_status IN ('Paid', 'Unpaid', 'Overdue')),
    FOREIGN KEY(shipment_id) REFERENCES Shipment(shipment_id)
);

CREATE TABLE TemperatureRecord (
    temperature_id INT PRIMARY KEY,
    shipment_id INT NOT NULL,
    recorded_at TIMESTAMP NOT NULL,
    temperature_degree DECIMAL(5,2) NOT NULL,
    temperature_status VARCHAR(15) NOT NULL CONSTRAINT temperature_status_check
CHECK (temperature_status IN ('Within Range', 'Out of Range')),
    FOREIGN KEY (shipment_id) REFERENCES Shipment(shipment_id)

```

);

A few types of constraints were used to achieve data integrity. PRIMARY KEY ensures that every row is unique, and composite primary keys in SupplierCapability and ShipmentItem ensure that there are no duplicate rows in the bridge tables. FOREIGN KEY ensures the relationship between tables and avoids orphan records. UNIQUE constraints avoid the duplication of email addresses and ensure that every shipment has only one invoice. CHECK constraints ensure that only valid values are inserted into columns like storage_condition to be either Ambient, Cold or Frozen, and that the temperature range is compatible with the storage condition. The due_date column uses GENERATED ALWAYS AS, and it will calculate itself automatically as 15 days after the invoice date.

Task 5: Data Population Using SQL DML

The database was populated using SQL INSERT statements. The data was verified to ensure its accuracy and consistency with the business rules, such as that orders can be made only by Active customers, cold and frozen are only assigned to those suppliers who have the matching storage capability, and any shipment cannot use the Suspended supplier. The row counts are Customer (30), Supplier (15), SupplierCapability(41), Medicine (25), Orders (60), Shipment (80), ShipmentItem (116), Invoice(80), TemperatureRecord (89).

The main transaction table, Shipment, has 80 records in it; the dates range through six months (January to July 2026). The sample data has been made for this scenario and has been manually verified to ensure it makes sense for a pharmaceutical logistics company.

Task 6: Indexing

Index 1 – shipment_status (searching/filtering).

This column is a good choice for an index because it is commonly used to search for shipments by their status, such as Pending, In Transit, or Failed. These searches happen regularly and return only a few rows; therefore, an index allows them to speed up their execution by querying only specific rows instead of scanning all of them. The EXPLAIN (ANALYZE) query shows that the execution time was 0.064 ms.

SQL:

```
CREATE INDEX idx_shipment_status  
ON shipment (shipment_status);
```

Index 2 – supplier_id (joining/grouping).

This column is a good choice for indexing because it links each shipment to its supplier. It is used to join the Shipment and Supplier tables and to group shipments by supplier for carrier reports. PostgreSQL does not create indexes for foreign keys automatically, so adding an index will make those queries faster. The EXPLAIN (ANALYZE) query shows that the execution time was 0.156 ms.

SQL:

```
CREATE INDEX idx_shipment_supplier  
ON shipment (supplier_id);
```

Index 3 – payment_status and due_date (filtering + sorting).

The two columns are a good choice for indexing because they are first queried using the invoice payment status and then sorted according to the invoice due date. By putting these columns in one index, the database can perform both operations at once. The EXPLAIN (ANALYZE) query shows that the execution time was 0.080 ms.

SQL:

```
CREATE INDEX idx_invoice_status_due  
ON Invoice (payment_status, due_date);
```

Above all three columns are justified based on how the database is likely to be queried. Many columns are only displayed, such as `special_note`, and were not indexed, as this would add cost without improving searches.

Phase 3: SQL Queries for Data Retrieval and Reporting

Task 7: Update Query

Business question: Identify unpaid invoices that become overdue after their due date.

UPDATE Invoice

SET payment_status = 'Overdue'

WHERE payment_status = 'Unpaid'

AND due_date < CURRENT_DATE

RETURNING invoice_id, shipment_id, due_date, payment_status;

	invoice_id [PK] integer	shipment_id integer	due_date date	payment_status character varying (15)
1	7	7	2026-03-24	Overdue
2	16	16	2026-05-04	Overdue
3	30	30	2026-04-14	Overdue
4	33	33	2026-07-19	Overdue
5	39	39	2026-06-06	Overdue
6	46	46	2026-02-02	Overdue
7	47	47	2026-04-07	Overdue
8	53	53	2026-04-22	Overdue
9	54	54	2026-07-02	Overdue
10	56	56	2026-02-17	Overdue
11	67	67	2026-07-03	Overdue
12	79	79	2026-04-26	Overdue

Figure 2: Output from the update query

Explanation: This query changes 12 invoices from Unpaid to Overdue. This implements Business Rule 13. The WHERE clause only applies to invoices that are Unpaid and past their due date; therefore, paid invoices and invoices that are still within their 15-day terms are not selected. The query was run inside a transaction (BEGIN... ROLLBACK), so the change could be seen without changing the data permanently.

Task 8: Delete Query

Business question: Remove old cancellations that are no longer needed for further operation.

```
DELETE FROM Orders o
```

```
WHERE o.order_status = 'Cancelled'
```

```
AND NOT EXISTS (SELECT 1 FROM SHIPMENT AS s WHERE o.order_id=s.order_id)
```

```
RETURNING o.order_id, o.customer_id, o.order_date, o.order_status;
```

	order_id [PK] integer	customer_id integer	order_date date	order_status character varying (20)
1	3	10	2026-02-10	Cancelled
2	17	16	2026-06-29	Cancelled
3	27	1	2026-01-24	Cancelled
4	43	6	2026-04-07	Cancelled
5	47	29	2026-02-24	Cancelled

Figure 3: Output from the DELETE query

Explanation: This query deletes 5 records from the Orders table. The NOT EXISTS condition is necessary because it avoids deleting a cancelled order with shipment records, which would break the foreign key and lose delivery history. This query only deletes obsolete orders, and only if they have been canceled and have no shipments.

Task 9: Filtering and Sorting Query

Business question: Which invoices are still outstanding, soonest due first?

```
SELECT
```

```
    invoice_id, shipment_id, due_date, total_amount, payment_status
```

```
FROM Invoice
```

```
WHERE payment_status='Overdue'
```

```
ORDER BY due_date DESC;
```

	invoice_id [PK] integer	shipment_id integer	due_date date	total_amount numeric (10,2)	payment_status character varying (15)
1	18	18	2026-07-08	6110.76	Overdue
2	14	14	2026-07-03	3042.29	Overdue
3	69	69	2026-07-02	7661.14	Overdue
4	27	27	2026-06-27	8400.38	Overdue
5	9	9	2026-06-24	2628.93	Overdue
6	36	36	2026-06-15	6269.91	Overdue
7	58	58	2026-06-14	1627.62	Overdue
8	29	29	2026-05-23	8599.75	Overdue
9	20	20	2026-05-16	5988.68	Overdue
10	57	57	2026-05-06	6082.84	Overdue

Figure 4: Output from the FILTERING AND SORTING query

Explanation: This query retrieves the overdue invoices sorted with the latest due date first. The WHERE clause filters only invoices flagged Overdue, and ORDER BY due_date DESC orders by due date, creating a follow-up list for the finance team. Its output is that it returns 24 overdue invoices on the fresh database. But if Task 7 has already run without a ROLLBACK query, it returns 36 because that update turns 12 unpaid invoices into overdue ones.

Task 10: Aggregation Query

Business question: Summarise overall shipment activity and performance.

SELECT

COUNT(*) AS Total_Shipments,

COUNT(*) FILTER (WHERE shipment_status = 'Delivered') AS Delivered_Shipments,

COUNT(*) FILTER (WHERE actual_delivery_date IS NULL AND shipment_status <> 'Failed') AS Not_delivered_yet_Shipments,

COUNT(*) FILTER (WHERE actual_delivery_date > expected_delivery_date) AS Late_Delivered_Shipments,

ROUND(AVG(actual_delivery_date - shipment_date)

FILTER (WHERE actual_delivery_date IS NOT NULL), 2) AS Avg_Delivered_days

FROM shipment;

	total_shipments bigint	delivered_shipments bigint	not_delivered_yet_shipments bigint	late_delivered_shipments bigint	avg_delivered_days numeric
1	80	58	17	28	4.86

Figure 5: Output from the Aggregation query

Explanation: This query summarizes all 80 shipments in one row. COUNT with FILTER splits them into groups: 58 delivered, 17 in transit, 28 delivered late and AVG gives the average transit time at 4.86 days. This average is also the derived attribute `delivery_duration`.

Task 11: Grouping Query

Business question: How is shipment delivered across transport types, and how many in each type have been delivered?

SELECT

 transportation_type, COUNT(*) AS shipments,

 COUNT(*) FILTER (WHERE shipment_status = 'Delivered') AS delivered

FROM Shipment

GROUP BY transportation_type

ORDER BY shipments DESC;

	transportation_type character varying (20)	shipments bigint	delivered bigint
1	Road	55	39
2	Air	11	9
3	Rail	11	7
4	Sea	3	3

Figure 6: Output from the Grouping query

Explanation: This query groups the 80 shipments by the type of transport. It counts how many shipments are in each type of transport along with how many of the shipments were actually delivered. The query shows that Road transport is used the most, with 55 shipments, and 39 of these shipments were delivered. It also showed that Rail transport and Air transport were used 11 shipments on each, and Sea transport was only 3 shipments.

Task 12: Pattern Matching Query

Business question: Find every Insulin product in the medicine.

SELECT

 medicine_id, medicine_name, medicine_category, storage_condition

FROM medicine

WHERE medicine_name ILIKE 'Insulin%'

ORDER BY medicine_name;

	medicine_id [PK] integer 	medicine_name character varying (100) 	medicine_category character varying (50) 	storage_condition character varying (10) 
1	6	Insulin Aspart	Diabetes	Cold
2	5	Insulin Glargine	Diabetes	Cold
3	7	Insulin Lispro	Diabetes	Cold

Figure 7: Output from the Pattern Matching query

Explanation: This query uses ILIKE to find every medicine whose name starts with "Insulin". ILIKE ignores upper and lower case, and the % after the word matches anything that follows. It returns three results, such as Insulin Aspart, Insulin Glargine, and Insulin Lispro. And resulted all Diabetes medicines that need Cold storage at 2–8 °C.

Task 13: Three-Table Join Query

Business question: For every temperature-sensitive medicine shipped, confirm that the assigned carrier holds the exact matching storage capability (Rule 8).

SELECT

 sh.shipment_id, sp.supplier_id, sp.supplier_name, m.medicine_id, m.medicine_name,
 m.storage_condition, spc.capability_type

FROM shipment AS sh

JOIN shipmentitem As si

 ON si.shipment_id = sh.shipment_id

JOIN medicine AS m

```

        ON m.medicine_id = si.medicine_id

JOIN supplier AS sp

        ON sp.supplier_id = sh.supplier_id

JOIN suppliercapability AS spc

        ON spc.supplier_id = sp.supplier_id

        AND spc.capability_type IN ('Cold Storage', 'Frozen Storage')

WHERE m.storage_condition <> 'Ambient'

        AND ( (m.storage_condition = 'Cold'AND spc.capability_type = 'Cold Storage')

        OR (m.storage_condition = 'Frozen' AND spc.capability_type = 'Frozen Storage') )

ORDER BY m.storage_condition;

```

	shipment_id integer	supplier_id integer	supplier_name character varying (100)	medicine_id integer	medicine_name character varying (100)	storage_condition character varying (10)	capability_type character varying (30)
1	75	1	ColdChain Logistics Ltd	12	Filgrastim	Cold	Cold Storage
2	68	6	NordFrost Transport	9	Hepatitis B Vaccine	Cold	Cold Storage
3	68	6	NordFrost Transport	11	Human Albumin 20%	Cold	Cold Storage
4	69	12	GreenLine Logistics	5	Insulin Glargine	Cold	Cold Storage
5	69	12	GreenLine Logistics	8	Influenza Vaccine	Cold	Cold Storage
6	73	1	ColdChain Logistics Ltd	8	Influenza Vaccine	Cold	Cold Storage
7	75	1	ColdChain Logistics Ltd	9	Hepatitis B Vaccine	Cold	Cold Storage
8	12	2	PharmaFreight Express	9	Hepatitis B Vaccine	Cold	Cold Storage
9	15	12	GreenLine Logistics	7	Insulin Lispro	Cold	Cold Storage
10	15	12	GreenLine Logistics	9	Hepatitis B Vaccine	Cold	Cold Storage
11	22	6	NordFrost Transport	6	Insulin Aspart	Cold	Cold Storage
12	22	6	NordFrost Transport	8	Influenza Vaccine	Cold	Cold Storage
13	30	12	GreenLine Logistics	5	Insulin Glargine	Cold	Cold Storage
14	30	12	GreenLine Logistics	12	Filgrastim	Cold	Cold Storage
15	36	4	Atlantic Air Cargo	11	Human Albumin 20%	Cold	Cold Storage
16	64	4	Atlantic Air Cargo	12	Filgrastim	Cold	Cold Storage
17	5	6	NordFrost Transport	1	Covid-19 mRNA Vaccine	Frozen	Frozen Storage
18	5	6	NordFrost Transport	2	Varicella Vaccine	Frozen	Frozen Storage
19	65	6	NordFrost Transport	1	Covid-19 mRNA Vaccine	Frozen	Frozen Storage
20	13	14	Arctic Cold Freight	1	Covid-19 mRNA Vaccine	Frozen	Frozen Storage
21	65	6	NordFrost Transport	3	Zoster Vaccine	Frozen	Frozen Storage
22	2	6	NordFrost Transport	1	Covid-19 mRNA Vaccine	Frozen	Frozen Storage

Figure 8: Output from the FIVE-TABLE JOIN query

Explanation: This query joins five tables to check the cold chain. It checks whether each of the temperature-sensitive medicines has been assigned the appropriate storage type

required by the supplier. There are 22 rows returned, and all the records show correct assignments, which indicate compliance with Business Rule 8. In the query, the WHERE clause is needed because without it, a supplier that has both storage types would come out as false matches.

Task 14: View Creation

Business question: A reusable GDP compliance view.

```
DROP VIEW IF EXISTS vw_cold_chain_compliance;
```

```
CREATE OR REPLACE VIEW vw_cold_chain_compliance AS
```

```
SELECT
```

```
    tr.shipment_id, sp.supplier_name, count(*) As readings_taken,
```

```
        count(*) filter (where tr.temperature_status='Out of Range')As breaches,
```

```
        ROUND(100.0* count (*) filter (where tr.temperature_status='Within Range')/  
count(*),1) As compliance_percentage
```

```
FROM temperaturerecord AS tr
```

```
JOIN shipment AS sh
```

```
    ON sh.shipment_id = tr.shipment_id
```

```
JOIN Supplier AS sp
```

```
    ON sp.supplier_id = sh.supplier_id
```

```
GROUP BY tr.shipment_id, sp.supplier_name;
```

Test the view: worst cold-chain shipments first.

```
SELECT * FROM vw_cold_chain_compliance
```

```
ORDER BY compliance_percentage ASC, shipment_id;
```

≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL Showing rows: 1 to 14					
	shipment_id integer	supplier_name character varying (100)	readings_taken bigint	breaches bigint	compliance_percentage numeric
1	68	NordFrost Transport	6	2	66.7
2	22	NordFrost Transport	6	1	83.3
3	2	NordFrost Transport	7	1	85.7
4	12	PharmaFreight Express	7	1	85.7
5	13	Arctic Cold Freight	7	1	85.7
6	15	GreenLine Logistics	7	1	85.7
7	5	NordFrost Transport	7	0	100.0
8	30	GreenLine Logistics	6	0	100.0
9	36	Atlantic Air Cargo	6	0	100.0
10	64	Atlantic Air Cargo	6	0	100.0
11	65	NordFrost Transport	6	0	100.0
12	69	GreenLine Logistics	6	0	100.0
13	73	ColdChain Logistics Ltd	6	0	100.0
14	75	ColdChain Logistics Ltd	6	0	100.0

Figure 9: Output from the Test of the VIEW query

This view provides the quality team a ready-made cold-chain query. It also calculates the number of temperature readings for each shipment, counts how many were out of range, and computes the percentage of compliance, which is also a derived attribute. The test of the view query running with `SELECT * FROM vw_cold_chain_compliance` returns 14 shipments; the lowest temperature compliance rate is for shipment 68 (NordFrost Transport), which has only 66.7% with 2 incidents.

Task 15: Analytics-Style Periodic Report Query

Business question: How did FastLink's monthly shipment volume and revenue change over the first half of 2026?

SELECT

```

    TO_CHAR(inv.invoice_date,'YYYY-MM')As report_month,
        COUNT(DISTINCT inv.shipment_id) As total_shipments,
        SUM(inv.total_amount)As monthly_revenue,

```

```

ROUND(AVG(inv.total_amount),2) As avg_invoice_value
FROM Shipment sh
JOIN Invoice inv on inv.shipment_id =sh.shipment_id
GROUP BY TO_CHAR(inv.invoice_date,'YYYY-MM')
ORDER BY report_month;

```

	report_month text	total_shipments bigint	monthly_revenue numeric	avg_invoice_value numeric
1	2026-01	8	30896.38	3862.05
2	2026-02	14	60569.32	4326.38
3	2026-03	9	42360.20	4706.69
4	2026-04	20	95015.34	4750.77
5	2026-05	11	55704.07	5064.01
6	2026-06	13	67570.91	5197.76
7	2026-07	5	18659.69	3731.94

Figure 10: Output from the Analytical Report query

This query builds a monthly report by using the function TO_CHAR that formats every invoice date as "YYYY-MM". Using the GROUP BY function brings invoices in the same month together, and the use of aggregate functions builds a summary for those invoices where COUNT returns the number of shipments invoiced, SUM returns the total revenue earned, and AVG returns the average invoice value. The output includes one record per month from January to July 2026, and April is the highest month in terms of shipments with 20 invoices and revenue of €95,015.

References

- [1] J. J. Liu, S. C. So, K. L. Choy, H. C. W. Lau, and S. K. Kwok, "Performance improvement of third-party logistics providers – an integrated approach with a logistics information system," *International Journal of Technology Management*, vol. 42, no. 3, pp. 226-249.
- [2] "Third Party Logistics (3PL)," Uniphar, [Online]. Available: <https://uniphar.com/globalsourcing/third-party-logistics-3pl>. [Accessed 24 July 2026].
- [3] J. Singh, "Lecture 3: Normalization".
- [4] "PostgreSQL Tutorial," [Online]. Available: <https://www.pgtutorial.com/>. [Accessed 24 July 2026].